Binary Tree

October 23, 2026

1 Constructing Binary Tree

[ ]:
```
class BinaryTree:
    def __init__(self, data):
        self.data = self.data
        self.left = None
        self.right = None

root = BinaryTree(1)
root.left = BinaryTree(2)
root.right = BinaryTree(3)
```

2 Printing Binary Tree

[4]:
```
class BinaryTree:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = BinaryTree(1)
root.left = BinaryTree(2)
root.right = BinaryTree(3)

def print_binary_tree(root):
    if root is None:
        return

    ## Format: Node: L-> LeftChildren, R-> RightChildren
    print(root.data, end=": ")

    if (root.left is not None):
        print(f"L-> {root.left.data}", end=", ")
    else:
        print(f"L-> None", end=", ")
```

```
    if root.right is not None:
        print(f"R-> {root.right.data}")
    else:
        print(f"R-> None")

    ## Calling the recursion for the left and right tree

    print_binary_tree(root.left)
    print_binary_tree(root.right)

print_binary_tree(root)
```

```
1: L-> 2, R-> 3
2: L-> None, R-> None
3: L-> None, R-> None
```

3 Taking input of a Binary Tree

[ ]:
```
class BinaryTree:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def take_input_binaryTree_recursion():
    data = int(input("Enter the data for the Node: "))
    if data==-1:
        return None

    node = BinaryTree(data)
    print(f"Enter the left child of {data}: ")
    node.left = take_input_binaryTree_recursion()

    print(f"Enter the right child of {data}: ")
    node.right = take_input_binaryTree_recursion()

    return node


def print_binary_tree(root):
    if root is None:
        return
    print(root.data, end=": ")
    if root.left is not None:
        print(f"L-> {root.left.data}", end=", ")
    else:
        print(f"L-> None", end=", ")
```

```
    if root.right is not None:
        print(f"R-> {root.right.data}")
    else:
        print(f"R-> None")

    print_binary_tree(root.left)
    print_binary_tree(root.right)

print("Enter the binary Tree data (-1 for no node)")
root = take_input_binaryTree_recursion()
print_binary_tree(root)
```

```
Enter the binary Tree data (-1 for no node)
Enter the left child of 1:
Enter the left child of 2:
Enter the left child of 3:
Enter the right child of 3:
Enter the right child of 2:
Enter the left child of 4:
Enter the right child of 4:
Enter the right child of 1:
Enter the left child of 5:
Enter the left child of 6:
Enter the right child of 6:
Enter the right child of 5:
Enter the left child of 7:
Enter the right child of 7:
1: L-> 2, R-> 5
2: L-> 3, R-> 4
3: L-> None
R-> None
4: L-> None
R-> None
5: L-> 6, R-> 7
6: L-> None
R-> None
7: L-> None
R-> None
```

4 Taking the input level wise

[2]:
```
from collections import deque

class binarytree:
    def __init__(self, data):
        self.data = data
```

```
        self.left = None
        self.right = None

def take_input_level_wise():
    data = int(input("Enter the data for the Root: "))
    if data==-1:
        return None

    root = binarytree(data)
    queue = deque([root])

    while queue:
        current_node = queue.popleft()

        left_child_data = int(input(f"Enter left child for {current_node.data}:␣
 ↪"))
        if left_child_data !=-1:
            left_node = binarytree(left_child_data)
            current_node.left = left_node
            queue.append(left_node)

        right_child_data = int(input(f"Enter the right child for {current_node.
 ↪data}: "))
        if right_child_data!=-1:
            right_node = binarytree(right_child_data)
            current_node.right = right_node
            queue.append(right_node)

    return root

def print_binary_tree(root):
    if root is None:
        return
    print(root.data, end=": ")
    if root.left is not None:
        print(f"L-> {root.left.data}", end=", ")
    else:
        print(f"L-> None", end=", ")

    if root.right is not None:
        print(f"R-> {root.right.data}")
    else:
        print(f"R-> None")

    print_binary_tree(root.left)
    print_binary_tree(root.right)
```

```
print("Enter the Binary Tree data (-1 for No Node)")
root = take_input_level_wise()

print_binary_tree(root)
```

```
Enter the Binary Tree data (-1 for No Node)
1: L-> 2, R-> 3
2: L-> 2, R-> 5
2: L-> None, R-> None
5: L-> None, R-> None
3: L-> 4, R-> None
4: L-> None, R-> None
```

5 Daimeter of Tree

- it is the maximum distance between one node to another

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]: